

Product Requirements Document: "Aura" - Open-Source Brand Intelligence Platform

Vision and Strategic Context

Product Overview & Vision

The vision for Aura is to democratize brand intelligence by providing small businesses, developers, and marketers with a powerful, insightful, and completely free social listening tool.

Aura is a Python-based web application designed to aggregate and analyze public online conversations pertaining to a specific company or brand. Its core function is to distill vast quantities of unstructured text from disparate online sources into a coherent and actionable intelligence dashboard. The platform identifies public sentiment, highlights recurring themes in customer feedback, and provides a holistic, data-backed view of a brand's public perception. A foundational tenet of the Aura platform is its commitment to open-source principles and cost-free operation; it achieves its objectives without relying on expensive, proprietary platforms or paid API subscriptions.

The entire technology stack is predicated on this principle. The user interface and web framework will be built using **Streamlit**, an open-source Python library that enables the rapid development of data-centric applications.¹ Initial data persistence and management will be handled by

SQLite, a self-contained, serverless SQL database engine that is included by default with Python, requiring no separate installation or configuration.³ The data acquisition and analysis layers will be composed of a curated suite of powerful, free-to-use

Python libraries. These include:

- **Hugging Face Transformers** for state-of-the-art sentiment analysis.⁴
- **google-play-scraper** for comprehensive review extraction from the Google Play Store.⁵
- **app-store-web-scraper** for review extraction from the Apple App Store.⁶
- **PRAW (The Python Reddit API Wrapper)** for accessing discussions on Reddit.⁷
- **pytrends** for analyzing public search interest via Google Trends.⁹
- **instaloader** for experimental data collection from Instagram.¹¹

Problem Statement & User Value

Market intelligence and social listening have become indispensable components of modern business strategy. Understanding customer sentiment, tracking brand reputation, and analyzing competitor strategies are no longer optional activities for growth-oriented organizations. However, the professional-grade tools that provide these capabilities, such as Brandwatch or Sprinklr, come with enterprise-level price tags that are prohibitively expensive for a significant segment of the market. Startups, small-to-medium businesses (SMBs), independent developers, and non-profit organizations are often priced out, leaving them unable to systematically leverage public data to inform their decisions. They are relegated to manual, time-consuming, and often incomplete methods of gathering feedback, which are neither scalable nor quantifiable.

Aura directly addresses this market gap by providing a zero-cost entry point into the world of data-driven brand management. The value delivered to the user is immediate and substantial, empowering them to:

- **Gain Actionable Insights:** Move beyond anecdotal evidence to understand what customers are saying at scale about their brand, products, and services.
- **Prioritize Effectively:** Identify and quantify the most common themes of positive and negative feedback, enabling product development teams to prioritize bug fixes, feature enhancements, and strategic initiatives based on direct market signals.
- **Track Brand Health:** Monitor the overall sentiment and visibility of their brand over time, providing a crucial metric for the success of marketing and public relations efforts.
- **Ensure Full Transparency:** Access the raw, source-level data that underpins

every analysis. This traceability builds trust in the platform's conclusions and empowers advanced users to conduct their own deep-dive investigations.

Core Principles

The development and evolution of the Aura platform will be guided by a set of unwavering core principles that define its identity and purpose.

- **Radical Accessibility:** The core functionality of the tool must always be free to use. The architecture will be based on replicable, well-documented open-source components, ensuring that the platform is not only free to use but also transparent in its construction.
- **Data-Driven, Not Data-Overwhelming:** The primary objective is not merely to aggregate data but to distill it. The user interface and analysis modules will be designed to transform a firehose of unstructured text into simple, digestible, and actionable insights. The guiding design question will always be how to clearly answer, "What is working for this brand?" and "What is not working?".
- **Transparent & Traceable:** Every insight, metric, and theme presented on the dashboard must be directly and easily traceable back to its source data. A user should never have to question the origin of an analysis. By providing a clear path from a high-level sentiment score down to the individual review or comment that contributed to it, Aura builds user trust and empowers deeper, more nuanced exploration.⁴
- **Pragmatic Extensibility:** The system architecture must be modular and resilient. It must acknowledge that different data sources have different levels of reliability; official APIs are more stable than unofficial web scrapers. This principle dictates an architecture where data harvester modules are isolated, allowing the system to function even if one source becomes temporarily or permanently unavailable. This approach ensures long-term viability and simplifies the process of adding new data sources in the future.

User Personas and Core Journey

Primary Persona: "Maria", the SMB Marketer

- **Biography:** Maria is the sole marketing professional at a 30-person company that has developed a popular consumer gadget, "GlowWidget," which is supported by a companion mobile app. She is responsible for all aspects of marketing, from social media and content creation to public relations and customer engagement.
- **Goals:** Maria's primary objectives are to increase brand loyalty, improve the company's mobile app ratings on both the Google Play and Apple App Stores, and provide the product development team with concrete, data-backed customer feedback to guide future iterations of the GlowWidget and its software.
- **Pain Points:** Maria operates with a minimal budget, which completely excludes the possibility of licensing enterprise-grade social listening software. Her current process involves spending several hours each week manually searching Twitter, browsing relevant subreddits, and reading through app store reviews. She attempts to copy and paste key comments into a spreadsheet, but this process is inefficient, prone to missing important trends, and produces qualitative data that is difficult to present as a quantifiable business case to leadership.
- **Aura's Role:** Aura automates Maria's entire manual workflow. It provides her with a centralized dashboard where she can, in minutes, see a quantitative overview of brand sentiment. The "What's Working" and "What's Not Working" sections give her immediate, shareable insights. She can use the direct quotes from the source data explorer to enrich her marketing reports and provide the product team with specific, actionable examples of customer praise and frustration.

Secondary Persona: "David", the Indie App Developer

- **Biography:** David is a talented solo developer who has recently launched a new productivity application, "TaskFlow," on both the Google Play Store and the Apple App Store. He is responsible for all coding, marketing, and user support.
- **Goals:** David's immediate goal is to iterate on his app quickly. He needs to identify bugs reported by users and understand which feature requests are most in-demand to prioritize his limited development time effectively.
- **Pain Points:** Managing feedback is a significant bottleneck. He has to constantly switch between the Google Play Console and Apple's App Store Connect to read

and categorize reviews. This context-switching is inefficient, and because he is focused on responding to individual comments, he often misses the larger, overarching themes emerging from the user base as a whole.

- **Aura's Role:** Aura provides David with a unified command center for all his app feedback. It aggregates reviews from both stores into a single, searchable interface. The sentiment analysis engine automatically flags highly negative reviews, which he can quickly scan for potential bug reports. Conversely, it highlights highly positive reviews, which often contain validation for his feature choices and ideas for future development.

The Core User Journey

The ideal user experience is designed to be intuitive, guiding the user from a simple query to deep insights with minimal friction.

1. **Initiation:** The user, for example Maria, lands on the Aura homepage. The design is minimalist and focused, presenting a single call to action: a text input field with the label, "Enter a company name to begin."
2. **Validation:** Maria types "GlowWidget" into the field and clicks "Analyze." In the background, Aura's backend sends this query to a free-tier company enrichment API, such as TheCompaniesAPI or Abstract API.¹³ These services return structured data about potential matches. The Aura interface updates to present Maria with a clear choice: "Did you mean: 1. GlowWidget Inc. (glowwidget.com), 2. Glow Widget LLC (glow-widget.io)". This step is crucial for disambiguation, ensuring that the subsequent analysis targets the correct entity. Maria selects "GlowWidget Inc."
3. **Processing:** The application transitions to a loading state that provides transparent feedback on the work being performed. Maria sees messages like: "Searching for GlowWidget Inc. on Google Play...", "Analyzing 1,500 reviews...", "Fetching discussions from Reddit...", and "Calculating sentiment...". This keeps the user engaged and informed during the data collection process.
4. **Insight Discovery:** After a short wait, the "Aura Dashboard" loads. The most prominent information is a set of high-level metrics at the top of the page: "Overall Sentiment: 72% Positive." Below this, the screen is divided into two clear columns. The "What's Working" column lists top positive themes identified from the data, such as "easy to use," "great design," and "long battery life." The "What's Not Working" column lists negative themes like "frequent crashes," "poor

connectivity," and "confusing UI."

5. **Deep Dive:** Maria is concerned about the "frequent crashes" theme. She clicks on it. The "Source Data Explorer" table at the bottom of the page instantly filters to show only the raw reviews and comments that were categorized under this theme. She can now read the full text of each piece of user feedback, see its original source (e.g., Google Play, Reddit), the date it was posted, and its individual sentiment score. This fulfills the core principle of traceability.
6. **Contextualization (Phase 2 Feature):** To understand the broader context, Maria navigates to the "Brand Trends" tab. Here, she sees a line chart generated using data from pytrends.⁹ The chart shows that search interest for "GlowWidget" has been steadily increasing over the past 90 days, which she can correlate with a recent marketing campaign she launched. This adds another layer of intelligence to her analysis.

Phased Feature Rollout

The development of Aura is structured into three distinct phases. This approach allows for the rapid delivery of core value in a Minimum Viable Product (MVP), followed by iterative expansion of capabilities based on the platform's foundational architecture.

Phase 1: Minimum Viable Product (MVP) - Foundational Insights

The primary goal of the MVP is to deliver on the core promise of the application: to provide actionable brand intelligence from a reliable, high-volume set of data sources. This phase focuses on establishing the end-to-end pipeline from company validation to dashboard presentation.

Epic 1.1: Company Onboarding & Validation

This epic encompasses the user's first interaction with Aura and is the critical gateway

to the entire analysis process. It is responsible for translating a user's potentially ambiguous input into a specific, verifiable corporate entity.

- **Technical Implementation:** The frontend will be built using Streamlit's `st.text_input` widget to capture the company name from the user.¹ Upon submission, the backend will execute an API call to a free-tier company data service. Primary candidates for this service are **TheCompaniesAPI**¹⁴ and **Abstract API**¹³, both of which offer generous free plans suitable for an MVP. The objective of this API call is not just to verify existence but to retrieve a list of potential company profiles, each including the official company name and, most importantly, its primary web domain. The returned list of potential matches will be presented to the user for selection via `st.radio` or a series of `st.button` widgets.
- **User Story:** "As a user, I want to enter a company name and be presented with a list of verified potential matches so that I can ensure the tool analyzes the correct entity."
- **Acceptance Criteria:**
 - The user can input a string into a designated text field on the main page.
 - The system successfully queries an external company data API upon submission.
 - A list of potential company matches, clearly displaying both the company name and its domain, is rendered on the screen.
 - If the external API returns no matches, a user-friendly message, such as "No companies found. Please try a different name or be more specific," is displayed.
 - The user's selection of a company from the list successfully triggers the subsequent data harvesting stage.
- **Architectural Significance:** The process of company validation serves a purpose far beyond simple user confirmation; it is a fundamental act of data normalization that underpins the reliability of the entire platform. A user's input of "Google" is ambiguous. An external API can clarify this, offering choices like "Alphabet Inc." (domain: abc.xyz) and "Google LLC" (domain: google.com). When the user selects "Google LLC," the system doesn't just store the name; it captures a structured data object: `{'name': 'Google LLC', 'domain': 'google.com'}`. This canonical name and domain become the primary keys for all subsequent searches. The system can now confidently search for the developer "Google LLC" on the app stores, look for an official "r/google" subreddit, or identify a YouTube channel associated with that specific brand identity. Without this validation step,

all downstream data collection would be based on a high-risk, ambiguous string search, leading to inaccurate and unreliable results. Therefore, the data object passed from this module to the data harvesters must be a structured entity containing at a minimum, the canonical name and domain.

Epic 1.2: Core Data Harvester (Google Play & Reddit)

This module is the workhorse of the MVP, responsible for collecting the initial corpus of text data from two of the most reliable and data-rich free sources available.

- **Technical Implementation:**

- **Google Play Store:** The system will leverage the google-play-scraper Python library, a robust tool for crawling the Play Store without external dependencies.⁵ The harvester will first use the library's search function, passing the canonical company name to identify the associated developer profile and list of published apps. For each relevant app, it will then invoke the reviews_all function to methodically retrieve all available user reviews.
 - **Reddit:** Data collection from Reddit will be handled by the **PRAW** library, the official Python wrapper for the Reddit API.⁷ The harvester will search for subreddits whose names are closely related to the company name (e.g., r/{company_name}, r/{product_name}). It will then scrape the top-rated and newest posts, along with all their associated comments, from the most relevant subreddit. This process will respect the Reddit API's limitation of returning a maximum of 1000 items per listing request.¹⁵
 - **Data Persistence:** All collected data—reviews, Reddit posts, and comments—will be persisted in a local **SQLite** database.³ The database schema will be designed for this purpose, with tables that include fields for source (e.g., 'Google Play'), content (the raw text), author, date, rating (where applicable), and a foreign key linking back to the company_id from the validation step.
- **User Story:** "As the system, once a company is validated, I need to automatically find and download all relevant user reviews from the Google Play Store and recent discussions from Reddit to create a dataset for analysis."
 - **Acceptance Criteria:**
 - The system can successfully identify and list applications published by the validated company on the Google Play Store.

- The system can retrieve and store all available reviews for the identified applications.
- The system can identify a relevant subreddit and retrieve the top 1000 posts and their corresponding comments.
- All collected data is correctly inserted into the SQLite database with proper source attribution and linked to the correct company.
- The harvesting process includes graceful error handling for scenarios such as no app being found on the Play Store or the relevant subreddit being private or non-existent.

Epic 1.3: Sentiment Analysis Engine

This module represents the core analytical intelligence of Aura. It is responsible for processing the raw, unstructured text collected by the harvesters and assigning a quantitative sentiment score to each entry.

- **Technical Implementation:** The engine will be built upon the transformers library from Hugging Face, a leading platform for open-source machine learning models.¹⁶ Specifically, it will load the `siebert/sentiment-roberta-large-english` model.⁴ This model is chosen for its exceptional performance; it was fine-tuned on 15 different datasets to enhance its generalization capabilities and achieves an average accuracy of 93.2%, significantly outperforming models trained on single-domain data.⁴ The model will be loaded using the convenient `pipeline("sentiment-analysis",...)` function.
To ensure optimal performance and prevent the costly reloading of this large model on every user interaction, it will be cached in memory using Streamlit's `@st.cache_resource` decorator. This decorator is specifically designed for caching global resources like ML models or database connections.¹⁷ The engine will read batches of text from the SQLite database, run them through the sentiment analysis pipeline, and then update the corresponding records in the database with the resulting sentiment label ('POSITIVE' or 'NEGATIVE') and the model's confidence score.
- **User Story:** "As the system, I need to analyze every piece of collected text and assign a positive or negative sentiment score so that the data can be aggregated and visualized."
- **Acceptance Criteria:**

- The system correctly downloads and loads the specified Hugging Face model upon application startup.
- Every text-based record retrieved by the harvesters is processed by the sentiment analysis engine.
- Each record in the database is updated with a sentiment label (e.g., 'POSITIVE') and a numerical confidence score.
- The analysis process is robust and can handle texts of varying lengths and quality without crashing.

Epic 1.4: The "Aura" Dashboard v1

This is the user-facing culmination of the entire pipeline, where raw data is transformed into visual, understandable intelligence. The entire UI will be constructed using Streamlit components to facilitate rapid development.¹

- **Technical Implementation:**

- **High-Level Metrics:** The top of the dashboard will feature key performance indicators displayed using `st.metric`. These will include "Overall Sentiment Percentage," "Total Mentions Analyzed," and "Positive/Negative Count."
- **What's Working/Not Working:** This core insight feature will be implemented by performing a simplified form of topic modeling. The system will isolate the text from the most positive (e.g., score > 0.9) and most negative (e.g., score < 0.1) comments. It will then identify the most frequent keywords or two-word phrases (bigrams) within each set. These themes will be displayed in two parallel columns created with `st.columns`, providing an at-a-glance summary of praise and criticism.
- **Source Data Explorer:** This component is critical for fulfilling the "Transparent & Traceable" principle. It will be implemented using `st.dataframe` to display the raw data from the SQLite database in a tabular format. The table will be interactive by default, allowing the user to sort by data source, date, sentiment score, or to search the content.
- **Performance Optimization:** All functions that load and aggregate data from the SQLite database for display on the dashboard will be wrapped in Streamlit's `@st.cache_data` decorator. This decorator caches the output of computations, ensuring that the UI remains fast and responsive, even when the user interacts with widgets that trigger a script rerun.¹⁷
- **User Story:** "As a user, I want to see a clear, high-level summary of the sentiment

surrounding a brand, with the ability to drill down into the specific data points that contribute to the analysis."

- **Acceptance Criteria:**

- The dashboard successfully displays an overall sentiment score as a percentage.
- The dashboard presents two distinct lists of key themes, one for positive feedback and one for negative feedback.
- A searchable and sortable table on the dashboard displays all the source data, including the comment text, original source, date, and assigned sentiment.
- As an ideal interaction, clicking on a theme in the "What's Working/Not Working" section filters the source data table to show only the comments that contributed to that theme.

Phase 2: Expanding the Aperture

With the foundational pipeline established, the goal of Phase 2 is to broaden the scope of data collection by integrating new platforms and adding a layer of macro-level context to the analysis. This phase focuses on demonstrating the platform's extensibility while carefully managing the unique constraints of new data sources.

Epic 2.1: Multi-Platform Data Expansion (Apple App Store & YouTube)

This epic integrates two highly valuable data sources: the Apple App Store and YouTube. The implementation must be architected with a deep understanding of the technical and policy-based limitations of accessing data from these platforms.

- **Technical Implementation:**

- **Apple App Store:** The harvester will use the app-store-web-scraper Python library.⁶ This library works by scraping the web version of the App Store. The harvester will search for the company's apps and retrieve user reviews. A critical constraint of this library is that it relies on an undocumented internal Apple API that provides a maximum of 10 pages of reviews, with up to 50

reviews per page. This imposes a

hard limit of 500 of the most recent reviews per app.⁶

- **YouTube:** Data from YouTube will be collected using the official **YouTube Data API v3**.¹⁹ To operate within the free tier's daily quota of 10,000 units²⁰, the implementation must be highly efficient. It will follow a "Search then Fetch" pattern. A single, initial search.list call (cost: 100 units) will be made to find a list of videos relevant to the brand. The system will then use the much cheaper commentThreads.list method (cost: 1 unit per video) to fetch the comments for each relevant video ID returned by the search.¹⁹
- **Architectural Significance:** This epic introduces the concept of asymmetric data depth and quota-driven architecture. The Apple scraper's 500-review limit and the YouTube API's daily quota are not mere inconveniences; they are hard constraints that must be reflected in the product's design and user interface to maintain transparency and trust. Failing to account for these limitations would lead to a poor user experience (e.g., the YouTube feature suddenly breaking mid-day) or misleading analysis (e.g., the volume of Apple feedback appearing artificially low). The product must be designed *around* these realities.
- **PRD Requirements:**
 - The user interface for displaying Apple App Store analysis **must** include a clear and visible disclaimer, such as: (Analysis based on the 500 most recent reviews available).
 - The YouTube integration module **must** track its own API quota usage internally. If the daily quota is exceeded, the feature must be gracefully disabled, and a message must be displayed to the user, such as: YouTube analysis is temporarily unavailable due to daily usage limits. Please try again tomorrow.

Epic 2.2: Macro Trend Analysis (Google Trends)

This epic adds a new dimension to the analysis by incorporating data on public search interest, providing a macro-level context for the micro-level sentiment data.

- **Technical Implementation:** The system will integrate the pytrends Python library, an unofficial API for Google Trends.⁹ On the dashboard, a new tab or expandable section titled "Brand Interest" will be created. The backend will use the pytrends.interest_over_time() method to retrieve historical search interest data for

the company's name, typically over the last 12 months. This time-series data will then be visualized as a clean line chart using Streamlit's native `st.line_chart` function. While `pytrends` is highly effective, it can be subject to rate-limiting by Google if used excessively, so results will be heavily cached.¹⁰

- **User Story:** "As a user, I want to see how public search interest in a brand has trended over time, so I can correlate it with sentiment changes or the impact of my marketing campaigns."
- **Acceptance Criteria:**
 - A new, clearly labeled section on the dashboard displays a line chart of Google Trends data for the selected company.
 - The chart is clearly labeled with the appropriate date range.
 - The feature handles cases where Google Trends has no data for a given search term.

Epic 2.3: Dashboard Enhancements v2

This epic focuses on upgrading the dashboard to elegantly incorporate the new data sources and prevent information overload.

- **Requirements:**
 - A primary filter dropdown menu will be added to the top of the dashboard. Its options will include "All Sources," "Google Play," "Reddit," "YouTube," and "Apple App Store."
 - Changing the selection in this filter must dynamically update all dashboard components: the high-level metrics, the "What's Working/Not Working" themes, and the Source Data Explorer table.
 - This provides the user with the ability to perform both a holistic analysis across all platforms and a focused analysis on a single source of feedback.

Phase 3: Advanced Capabilities & User Retention

The goal of Phase 3 is to introduce more advanced and experimental features that differentiate the product and encourage repeat usage. This phase embraces higher-risk data sources and adds functionality that builds long-term value for the

user.

Epic 3.1: Experimental Social Scraping (Instagram)

This epic adds Instagram as a data source, explicitly acknowledging its unofficial and potentially unstable nature.

- **Technical Implementation:** The system will use the instaloader library to attempt to find a company's public Instagram profile and download metadata and comments from its recent posts.¹¹ This feature requires extensive and robust error handling. Instagram employs sophisticated anti-scraping measures, and the implementation must be prepared to gracefully manage rate limits, login requests, and potential temporary IP blocks.¹¹
- **Architectural Significance:** This epic introduces the "risk spectrum" of data sources. instaloader is a reverse-engineered scraper, making it inherently more fragile than the official APIs used in earlier phases. A minor change to Instagram's frontend code could break the scraper at any moment. Presenting data from this source with the same level of confidence as API-sourced data would be misleading and would damage user trust when (not if) it fails. The product must frame this feature correctly to manage user expectations.
- **PRD Requirements:**
 - The Instagram data source will be clearly labeled as "**Experimental**" or "**Beta**" in all relevant UI elements (e.g., the source filter dropdown).
 - A prominent tooltip or info icon next to the "Experimental" label will provide a clear explanation: Instagram data is collected via an unofficial method and may be incomplete or unavailable at times.
 - The backend module for Instagram scraping must be fully isolated in the application's architecture. Its failure must be caught and handled without impacting the execution or data collection of any other harvester module.

Epic 3.2: Historical Snapshot & Comparison

This feature allows users to save the results of an analysis and compare them over time, creating historical context where none exists natively.

- **Architectural Significance:** This feature is a direct and clever workaround for the "recency bias" inherent in many free APIs. The Reddit API only provides the most recent 1000 items, and the Apple scraper only provides the 500 most recent reviews. True historical analysis by querying the past is impossible with these tools. The only way to track change over time is for Aura to create its own historical record. A user wants to know if their brand perception is improving. The system can't ask Reddit what sentiment was like six months ago. However, it can run a full analysis today, save the aggregated results (e.g., 72% positive, top negative theme: "frequent crashes"), and then allow the user to run another analysis a month from now. By comparing these two "snapshots," the system manufactures historical value that would otherwise be lost.
- **PRD Requirements:**
 - A "Save Report" button will be added to the main dashboard.
 - When clicked, this action will store the key aggregated metrics (overall sentiment, mention counts, etc.) and the top positive/negative themes in a new SQLite table, along with a timestamp and the associated company ID.
 - A new "Comparison View" page or mode will be created. In this view, a user can select two saved reports for the same company.
 - The UI will then display a side-by-side comparison of the metrics and themes from the two selected snapshots, highlighting changes over time.

Epic 3.3: User-Centric Features (Optional Extension)

This epic outlines a potential path toward personalization and user retention by introducing basic account functionality.

- **Requirements:**
 - Implementation of simple user registration and login. This could be handled by a third-party Streamlit component to accelerate development.
 - Saved companies and reports from Epic 3.2 would be linked to a unique user ID in the database.
 - This epic introduces the first real need for multi-user data isolation and signals that if the platform's usage scales, the data persistence layer will need to be migrated from the single-file SQLite database to a more robust, client-server database like PostgreSQL to handle concurrent users effectively.

System-Wide & Non-Functional Requirements

This section details the overarching technical requirements, constraints, and operational principles that apply to the entire Aura system.

Technology Stack Specification

The Aura platform will be constructed exclusively from the following open-source and free-tier technologies:

- **Web Framework:** Streamlit ¹
- **Data Persistence:** SQLite ³
- **Sentiment Analysis:** Hugging Face Transformers ⁴
- **Data Acquisition Libraries:**
 - google-play-scraper ⁵
 - app-store-web-scraper ⁶
 - PRAW ⁸
 - google-api-python-client (for YouTube Data API v3) ¹⁹
 - pytrends ⁹
 - instaloader ¹¹
- **Company Validation API:** TheCompaniesAPI ¹⁴ or Abstract API ¹³ (Free Tier)

Data Source & API Management

This is the most critical non-functional requirement for the platform's stability and longevity. The system must be architected as a responsible "good citizen" of the web, robustly handling the unique limitations of each data source.

- **Requirements:**
 - A centralized request management system must be implemented. This system will be capable of handling configurable delays between requests, retries with

exponential backoff for transient errors, and queuing of API calls to respect rate limits.

- Each data harvester module (for Google Play, Reddit, etc.) must be independently configurable with its own specific rate limit, delay, and retry settings.
- The system must implement comprehensive logging for all API calls and scraping activities, capturing both successful requests and errors to facilitate debugging.
- **Key Artifact: Data Source Management Matrix:** This matrix serves as the single source of truth for the operational constraints of every data source used by Aura. It codifies the findings from the research phase and provides a critical reference for the development team, ensuring that architectural decisions are made with a full understanding of each source's limitations. This prevents costly mistakes, such as treating an unstable scraper with the same confidence as an official API, or building an inefficient workflow that rapidly exhausts a free API quota.

Table 4.2.1: Data Source Management Matrix

Data Source	Tool/Library	Free Tier Limit / Cost	Data Cap / Bias	Risk/Instability	Mitigation Strategy	Relevant Sources
Google Play Store	google-play-scraper	N/A (Web Scraping)	Can fetch all available reviews, but can be slow for apps with millions of reviews.	Low	Implement configurable delays between paged requests to avoid temporary IP blocks. Cache results aggressively using @st.cache_data.	⁵
Apple App Store	app-store-web-scraper	N/A (Web Scraping)	Hard limit of 500 most	Medium (Relies on an	Cache results heavily.	⁶

			recent reviews per app/country.	undocumented Apple API that could change without notice).	Crucially, display a clear disclaimer in the UI about the 500-review data limit.	
Reddit	PRAW	100 queries/minute (when using OAuth authentication).	1000 items per listing (e.g., top 1000 posts). Strong recency bias.	Low (Official API wrapper).	Mandate the use of OAuth for authentication. Queue and throttle requests to stay safely under the 100/min limit. Design features (like snapshots) around the 1000-item and recency limits.	8
YouTube	YouTube Data API v3	10,000 units/day. Search cost: 100 units. Video details: 1 unit. Comment fetch: 1	No inherent data cap, but constrained by daily quota.	Low (Official Google API).	Implement a "Search then Fetch" workflow to minimize expensive search calls. Track daily	19

		unit.			quota usage internally. Gracefully disable the feature with a user-facing message if the quota is exceeded.	
Google Trends	pytrends	Unknown, but exists. Reports of blocking after ~1400 requests.	Data is normalized and relative, not absolute search volume.	Medium (Unofficial API; Google can change endpoints)	Implement long sleep intervals (e.g., 60 seconds) upon receiving a 429 (Too Many Requests) error. Cache all Trend results for at least 24 hours.	9
Instagram	instaloader	N/A (Web Scraping)	Public data only. Highly unpredictable and subject to being blocked.	High (Actively opposed by platform).	Isolate the module so its failure does not affect the rest of the app. Label the feature as "Experimental" in the UI. Use	11

					rotating user-agents and consider proxy rotation for any scaled use.	
Company Validation	TheCompaniesAPI	500 free credits/month.	Dependent on provider's database coverage.	Low	Cache domain lookups to avoid re-querying for the same company. Design a mechanism to potentially cycle through multiple free provider API keys if one is exhausted.	¹³

Performance, Caching, and Scalability

- Performance:** The perceived performance and responsiveness of the application are paramount to user satisfaction. Given that data collection and analysis can be time-consuming, aggressive caching is not an optional optimization but a core architectural requirement. The heavy and judicious use of Streamlit's caching decorators, specifically `@st.cache_data` for data-producing functions and `@st.cache_resource` for loading large, static assets like the sentiment model, is mandatory.¹⁷

- **Scalability:** The MVP is intentionally designed for a single-user, on-demand execution model. The choice of SQLite as the data persistence layer is a deliberate trade-off, prioritizing simplicity and zero-configuration setup over concurrent-user scalability. The PRD acknowledges that should the application evolve into a multi-user platform with features like scheduled analyses or persistent user accounts, the data persistence layer will represent a primary scaling bottleneck and will need to be migrated to a more robust client-server database system, such as PostgreSQL.

Legal and Ethical Considerations

The operation of the Aura platform must adhere to strict legal and ethical standards to ensure its long-term viability and responsible conduct.

- **Terms of Service Compliance:** The application will strictly adhere to the published Terms of Service for all official APIs it consumes, including those from Google (for YouTube) and Reddit.
- **Responsible Scraping:** For tools that rely on web scraping (google-play-scraper, app-store-web-scraper, instaloader), the application will be configured to maintain a responsible request rate to avoid placing undue load on the target services. Where applicable, it will respect the directives found in the robots.txt files of the target domains.
- **Data Privacy:** The application will only process data that is explicitly public. No attempts will be made to access private user information, circumvent login walls, or otherwise retrieve data not intended for public consumption. All analysis will be performed on aggregated, anonymized data wherever possible.

Works cited

1. Streamlit • A faster way to build and share data apps, accessed August 11, 2025, <https://streamlit.io/>
2. Deep Dive into Data Apps with Streamlit - DEV Community, accessed August 11, 2025, <https://dev.to/alexmercedcoder/deep-dive-into-data-apps-with-streamlit-3e40>
3. SQLite with Python - Tutorialspoint, accessed August 11, 2025, https://www.tutorialspoint.com/sqlite/sqlite_python.htm
4. siebert/sentiment-roberta-large-english · Hugging Face, accessed August 11, 2025, <https://huggingface.co/siebert/sentiment-roberta-large-english>
5. google-play-scraper · PyPI, accessed August 11, 2025,

- <https://pypi.org/project/google-play-scraper/>
6. app-store-web-scraper · PyPI, accessed August 11, 2025, <https://pypi.org/project/app-store-web-scraper/>
 7. Why can't we use the API free-tier? : r/apolloapp - Reddit, accessed August 11, 2025, https://www.reddit.com/r/apolloapp/comments/146a1tu/why_cant_we_use_the_api_freetier/
 8. API Update: Enterprise Level Tier for Large Scale Applications - Reddit, accessed August 11, 2025, https://www.reddit.com/r/redditdev/comments/13wsiks/api_update_enterprise_level_tier_for_large_scale/
 9. Exploring Database trends using Python pytrends (Google Trends) - Ora-lytics, accessed August 11, 2025, <https://oralytics.com/2020/11/09/exploring-database-trends-using-python-pytrends-google-trends/>
 10. pytrends · PyPI, accessed August 11, 2025, <https://pypi.org/project/pytrends/>
 11. Instaloder — Download Instagram Photos and Metadata, accessed August 11, 2025, <https://instaloder.github.io/>
 12. instaloder - PyPI, accessed August 11, 2025, <https://pypi.org/project/instaloder/>
 13. Free Company Enrichment API - Search by Name or Domain - Abstract API, accessed August 11, 2025, <https://www.abstractapi.com/api/company-enrichment>
 14. The Companies API - Enrichment API for Companies, accessed August 11, 2025, <https://www.thecompaniesapi.com/>
 15. Is it impossible to get more than 1000 data with the free reddit API? : r/learnpython, accessed August 11, 2025, https://www.reddit.com/r/learnpython/comments/1d1i94i/is_it_impossible_to_get_more_than_1000_data_with/
 16. Models - Hugging Face, accessed August 11, 2025, <https://huggingface.co/models?other=sentiment-analysis>
 17. Unleash the power of Streamlit. A faster way to build and share data... | by Jeremy Sapienza | Data Reply IT | DataTech | Medium, accessed August 11, 2025, <https://medium.com/data-reply-it-datatech/unleash-the-power-of-streamlit-9d6aafcc9dfb>
 18. Streamlit — A faster way to build and share data apps. - GitHub, accessed August 11, 2025, <https://github.com/streamlit/streamlit>
 19. Quota Calculator | YouTube Data API | Google for Developers, accessed August 11, 2025, https://developers.google.com/youtube/v3/determine_quota_cost
 20. Youtube API limits : How to calculate API usage cost and fix exceeded API quota | Phyllo, accessed August 11, 2025, <https://www.getphyllo.com/post/youtube-api-limits-how-to-calculate-api-usage-cost-and-fix-exceeded-api-quota>
 21. Google Trends API.ipynb - Colab, accessed August 11, 2025, https://colab.research.google.com/github/Tanu-N-Prabhu/Python/blob/master/Google_Trends_API.ipynb